United States Patent Application Publication	20250286740
Kind Code	A1
Publication Date	September 11, 2025
Inventor(s)	Fay; Björn et al.

DECOMPOSITION OF MASKED VALUES

Abstract

The disclosure relates to decomposition of masked values in a cryptographically secure digital signing system. Example embodiments include a method of decomposing mod 44 an N bit Boolean share input (b′.sup.B,k), where N>12, the method comprising: i) reducing (**302-305**) a number of bits in the Boolean share input (b′.sup.B,k) by adding a lower 11:0 bits of the input (b′.sup.B,k) to an upper portion of the input left shifted by 2 bits to provide a first intermediate result (t.sub.1.sup.B,13) having M bits; ii) reducing (**306-309**) a number of bits of the first intermediate result (t.sub.1.sup.B,13) by adding a lower 6:0 portion of the intermediate result to an upper portion left shifted by 2 bits and subtracted from a multiple of 44 to provide a second intermediate result (t.sub.3.sup.B,8); and iii) adjusting (**310-316**) the second intermediate result (t.sub.3.sup.B,8) by adding and/or subtracting 44 to provide an output (w.sub.1.sup.B,k′) having a value within an interval of 0:43, the output (w.sub.1.sup.B,k′) being a mod 44 representation of the input (b′.sup.B,k).

Inventors: **Fay; Björn (Brande-Hörnerkirchen, DE), Renes; Joost Roland ('s-Hertogenbosch, NL), Schneider; Tobias (Graz, AT), Bronchain; Olivier (Auderghem, BE)**

Applicant: **NXP B.V.** (Eindhoven, NL)

Family ID: **90362395**

Appl. No.: **19/055079**

Filed: **February 17, 2025**

Foreign Application Priority Data

EP	24161824.8	Mar. 06, 2024

Publication Classification

Int. Cl.: **H04L9/34** (20060101); **H04L9/32** (20060101)

U.S. Cl.:

CPC **H04L9/34** (20130101); **H04L9/3247** (20130101);

Background/Summary

FIELD

[0001] The disclosure relates to decomposition of masked values in a cryptographically secure digital signing system.

BACKGROUND

[0002] Digital security infrastructure relies on a range of efficient and secure cryptographic operations, including symmetric and asymmetric cryptography. Current asymmetric cryptography schemes include RSA and ECC, which are widely used in many applications, for example to enable secure symmetric key exchange and to perform secure digital signing. It is infeasible using conventional computer technology to break such schemes provided that a sufficiently long key is used. A 256-bit key, for example, is in practice unbreakable using brute force methods on any current or future conventional computing means. With the anticipated introduction of quantum computing, however, such schemes could become vulnerable to attack. Further cryptographic standards are therefore being developed that are designed to be resistant to quantum computing algorithms. Recent significant advances in quantum computing have accelerated research into post-quantum cryptography (PQC) schemes, i.e. cryptographic algorithms which run on classical computers but are believed to be still secure even when faced with an adversary having access to a quantum computer.

[0003] In 2022, NIST (the National Institute of Standards and Technology) chose Dilithium as the primary PQC digital signature standard for the future. Dilithium is a digital signature scheme that is strongly secure under chosen message attacks based on the hardness of lattice problems over module lattices. As with traditional cryptography, Dilithium provides sufficient “black-box” security, but physical implementations of the standard may still be vulnerable to physical attacks, such as side-channel analysis and fault-injection attacks. While prior work already presents some proposals for dedicated countermeasures for Dilithium against such attacks, the current state of the art is still much less mature compared to traditional cryptography and there is still room for improvement.

SUMMARY

[0004] According to a first aspect there is provided a computer-implemented method of decomposing mod 44 an N bit Boolean share input, where N>12, the method comprising: [0005] i) reducing a number of bits in the Boolean share input by adding a lower 11:0 bits of the input to an upper portion of the input left shifted by 2 bits to provide a first intermediate result having M bits; [0006] ii) reducing a number of bits of the first intermediate result by adding a lower 6:0 portion of the intermediate result to an upper portion left shifted by 2 bits and subtracted from a multiple of 44 to provide a second intermediate result; and [0007] iii) adjusting the second intermediate result by adding and/or subtracting 44 to provide an output having a value within an interval of 0:43, the output being a mod 44 representation of the input.

[0008] The method allows for a more efficient way to compute mod 44 on Boolean shares in a secure system. The method requires only basic bit manipulation such as shifting operations, together with standard Boolean masked addition and subtraction operations. Implementing the method in a digital signature scheme therefore requires minimal overhead and can assist with

reducing implementation, code size and verification overhead.
[0009] The adding and subtracting operations may be defined as arithmetic addition and subtraction of Boolean share inputs.
Step i) may comprise: [0010] ia) reducing the number of bits in the Boolean share input by adding the lower 11:0 bits of the input to an upper N−1:12 portion of the input left shifted by 2 bits to provide a third intermediate result having P bits, where P<N; and [0011] ib) reducing a number of bits in the third intermediate result by adding a lower 11:0 bits of the third intermediate result to an upper P−1:12 portion of the third intermediate result left shifted by 2 bits to provide the first intermediate result.
Step ii) may comprise: [0012] iia) subtracting the first intermediate result from an integer multiple of 44 to provide a fourth intermediate result; [0013] iib) left shifting an upper 6 bits of the fourth intermediate result by two bits and subtracting this from a lower 7 bits of the fourth intermediate result to provide a fifth intermediate result; [0014] iic) adding 44 to the fifth intermediate result from 44 to provide a sixth intermediate result; and [0015] iid) subtracting a lower 6 bits of the sixth intermediate result to provide the second intermediate result.
[0016] The integer multiple may be 6.
The number N, i.e. the initial number of bits in the N bit Boolean share input, may be 32.
[0017] The number of bits in the first intermediate result, M, may be 12.
[0018] According to a second aspect there is provided a computer-implemented method of performing a signing operation in a digital signature scheme, the method comprising: [0019] receiving an input to be digitally signed; [0020] performing a digital signing operation on the received input using a secret key; and [0021] providing a signature output from the digital signing operation, [0022] wherein the step of performing the digital signing operation incorporates the method according to the first aspect.
[0023] The digital signature scheme may be Dilithium.
[0024] According to a third aspect there is provided a computer system for performing a signing operation in a digital signature scheme, the system configured to: [0025] receive an input to be digitally signed; [0026] perform a digital signing operation on the received input using a secret key; and [0027] provide a signature output from the digital signing operation, [0028] wherein the system is configured to perform the digital signing operation incorporating decomposing mod 44 an N bit Boolean share input, where N>12, by: [0029] i) reducing a number of bits in the Boolean share input by adding a lower 11:0 bits of the input to an upper portion of the input left shifted by 2 bits to provide a first intermediate result having M bits; [0030] ii) reducing a number of bits of the first intermediate result by adding a lower 6:0 portion of the intermediate result to an upper portion left shifted by 2 bits and subtracted from a multiple of 44 to provide a second intermediate result; and [0031] iii) adjusting the second intermediate result by adding and/or subtracting 44 to provide an output having a value within an interval of 0:43, the output being a mod 44 representation of the input.
[0032] Step i) may comprise: [0033] ia) reducing the number of bits in the Boolean share input by adding the lower 11:0 bits of the input to an upper N−1:12 portion of the input left shifted by 2 bits to provide a third intermediate result having P bits, where P<N; and [0034] ib) reducing a number of bits in the third intermediate result by adding a lower 11:0 bits of the third intermediate result to an upper P−1:12 portion of the third intermediate result left shifted by 2 bits to provide the first intermediate result.
[0035] Step ii) may comprise: [0036] iia) subtracting the first intermediate result from an integer multiple of 44 to provide a fourth intermediate result; [0037] iib) left shifting an upper 6 bits of the fourth intermediate result by two bits and subtracting this from a lower 7 bits of the fourth intermediate result to provide a fifth intermediate result; [0038] iic) adding 44 to the fifth intermediate result from 44 to provide a sixth intermediate result; and [0039] iid) subtracting a lower 6 bits of the sixth intermediate result to provide the second intermediate result.

[0040] The integer multiple may be 6.
[0041] N may be 32.
[0042] According to a fourth aspect there is provided a computer program comprising instructions to cause a computer processor to perform the method according to the first aspect.
[0043] There may be provided a computer program, which when run on a computer, causes the computer to configure any apparatus, including a circuit, controller, sensor, filter, or device disclosed herein or perform any method disclosed herein. The computer program may be a software implementation, and the computer may be considered as any appropriate hardware, including a digital signal processor, a microcontroller, and an implementation in read only memory (ROM), erasable programmable read only memory (EPROM) or electronically erasable programmable read only memory (EEPROM), as non-limiting examples. The software implementation may be an assembly program.
[0044] The computer program may be provided on a non-transitory computer readable medium, which may be a physical computer readable medium, such as a disc or a memory device, or may be embodied as a transient signal. Such a transient signal may be a network download, including an internet download.
[0045] According to a fifth aspect there is provided a hardware converter configured to decompose mod 44 an N bit Boolean share input, where N>12, the hardware converter comprising: [0046] a first bit reduction module configured to reduce a number of bits in the Boolean share input by adding a lower 11:0 bits of the input to an upper portion of the input left shifted by 2 bits to provide a first intermediate result having M bits; [0047] a second bit reduction module configured to reduce a number of bits of the first intermediate result by adding a lower 6:0 portion of the intermediate result to an upper portion left shifted by 2 bits and subtracted from a multiple of 44 to provide a second intermediate result; and [0048] an adjustment module configured to adjust the second intermediate result by adding and/or subtracting 44 to provide an output having a value within an interval of 0:43, the output being a mod 44 representation of the input.
[0049] The first bit reduction module may be configured to: [0050] reduce the number of bits in the Boolean share input by adding the lower 11:0 bits of the input to an upper N−1:12 portion of the input left shifted by 2 bits to provide a third intermediate result having P bits, where P<N; and [0051] reduce a number of bits in the third intermediate result by adding a lower 11:0 bits of the third intermediate result to an upper P−1:12 portion of the third intermediate result left shifted by 2 bits to provide the first intermediate result.
[0052] The second bit reduction module may be configured to: [0053] subtract the first intermediate result from an integer multiple of 44 to provide a fourth intermediate result; [0054] left shift an upper 6 bits of the fourth intermediate result by two bits and subtract this from a lower 7 bits of the fourth intermediate result to provide a fifth intermediate result; [0055] add 44 to the fifth intermediate result from 44 to provide a sixth intermediate result; and [0056] subtract a lower 6 bits of the sixth intermediate result to provide the second intermediate result.
[0057] The integer multiple may be 6.
[0058] The number N, i.e. the initial number of bits in the N bit Boolean share input, may be 32.
[0059] The number of bits in the first intermediate result, M, may be 12.
[0060] These and other aspects of the invention will be apparent from, and elucidated with reference to, the embodiments described hereinafter.

Description

BRIEF DESCRIPTION OF DRAWINGS
[0061] Embodiments will be described, by way of example only, with reference to the drawings, in which:

[0062] FIG. **1** is a schematic visual representation of a first part of a proposed algorithm for demasking a Boolean masked input mod 44;
[0063] FIG. **2** is a schematic visual representation of a second part of the proposed algorithm;
[0064] FIG. **3** is a flow diagram illustrating the proposed algorithm;
[0065] FIG. **4** is a schematic diagram illustrating a system for secure digital signing an input; and
[0066] FIG. **5** is a schematic diagram illustrating an example hardware converter for decomposing mod 44 an N bit Boolean share input.
[0067] It should be noted that the Figures are diagrammatic and not drawn to scale. Relative dimensions and proportions of parts of these Figures have been shown exaggerated or reduced in size, for the sake of clarity and convenience in the drawings. The same reference signs are generally used to refer to corresponding or similar feature in modified and different embodiments.
DETAILED DESCRIPTION
[0068] One critical operation of Dilithium that needs to be protected against attacks relates to the decomposition of masked polynomials. Existing solutions rely on different decomposition approaches for each security level of Dilithium (known as security levels 2, 3 and 5), resulting in higher implementation, code, and verification overhead to support all levels.
[0069] The scheme used in Dilithium requires operations involving arithmetic with polynomials having integer coefficients. When implemented, the main computationally expensive operations involve arithmetic operations with polynomials. Such computations are carried out in a ring R.sub.q=(custom-character/qcustom-character)[X]/(X.sup.256+1), i.e. the ring where polynomial coefficients are in custom-characterZ/qcustom-character and the polynomial arithmetic is performed modulo X.sup.256+1.
[0070] A signing operation of a digital signature scheme generates a signature for a given message using a secret key. If this secret key were to be leaked, it would invalidate the security properties provided by the scheme. It has been shown in [6] that unprotected implementations of Dilithium are vulnerable to implementation attacks, e.g., side-channel analysis. In particular, it has been demonstrated that the secret key can be extracted from physical measurements of key-dependent parts in the signing operation.
[0071] For Dilithium, the key-dependent operations include the decomposition of polynomials in a base a. In particular, for a coefficient a∈custom-character/qcustom-character, the decompose operation computes the high part a.sub.1 and the low part a.sub.0 such that a mod q=a.sub.1×α+a.sub.0 with
[00001]$-\frac{}{2} < a_0 \leq \frac{}{2}$
except if
[00002]$a_1 = \frac{q-1}{}$
where a.sub.1 is set to 0 and a.sub.0=(a mod q)−q, with
[00003]$-\frac{}{2} \leq a_0 < 0.$
The possible values for the decomposition base α are
[00004]$\frac{q-1}{16}$ and $\frac{q-1}{44}$
depending on the security level of Dilithium. Additionally, the parameter γ is defined such that α=2γ. While the decomposition operation is trivial in the unmasked case, a secure implementation of this digital signature scheme requires the integration of dedicated countermeasures for this step.
Countermeasures
[0072] Masking is a common countermeasure to thwart side-channel analysis and has been utilized for various applications [8]. Besides security, efficiency is also an important aspect when designing a masked algorithm. Important metrics for software implementations of masking are the number of operations and the number of fresh random elements required for the masking scheme.
Decomposition of Masked Polynomials
[0073] The first masking approach for Dilithium was proposed in [6] with the decomposition operation for prime modulus performed using multiple arithmetic additions modulus q over

Boolean shares. This approach takes as an input an arithmetic sharing of coefficients and produces Boolean-shared decompositions.

[0074] Azouaoui et al. [1] proposed an improved approach, which reduced the performance and randomness overhead compared to [6]. For security levels 3 and 5, their method relies on the fact that mod 16 can be efficiently computed on Boolean shares, given that 16=24. For security level 2, however, it is required to compute mod 44 instead, which is not a power of 2. Since this is not straightforwardly possible, Azouaoui et al. propose an alternative solution using a complex masked compression algorithm for this security level. Therefore, to support all security levels, an implementation needs to integrate two quite different decompose approaches.

[0075] To overcome this issue, Coron et al. [4] proposed two slightly tweaked decompose algorithms. The first one is based on the compressions approach which is extended to all security levels. The second one is based on the protected mod computation for which they utilize a complete SecB2AModp.sub.q.sup.d(x.sup.B,k) conversion. While these indeed work for all security levels, they rely on complex gadgets (compress) and require level-specific variants (e.g., SecB2AModp.sub.16.sup.d(x.sup.B,k) and SecB2AModp.sub.44.sup.d(x.sup.B,k)).

[0076] As described above, an implementation of all security levels of Dilithium may use efficient but significantly different approaches for each level [1], rely on complex gadgets which for example require division [4], or may require level-specific variants for each level, which again increases the implementation, code size and verification complexity [4].

[0077] A problem with existing solutions is the lack of an efficient method to compute mod 44 on Boolean shares. Such a solution would enable the implementation for level 2 decompose alongside the most efficient level 3 and 5 approach with only minimal changes.

[0078] Described herein is a method for computing mod 44 on Boolean shares securely and efficiently. The method does not require a complete SecB2AModp.sub.44.sup.d(x.sup.B,k) conversion. Instead, the method uses only basic bit manipulation such as a shift operation and standard Boolean masked addition or subtraction gadgets, the latter of which should already be implemented for other masked gadgets according to the Dilithium standard. The implementation overhead of the proposed method is therefore minimal.

[0079] Also disclosed is an implementation of how the masked mod gadget may be composed in an improved decompose algorithm, which supports all security levels of Dilithium while only requiring minimal changes between them. Overall, the method helps to reduce the implementation, code size, and verification overhead compared to existing solutions.

[0080] In the following, the notation and existing gadgets are introduced, followed by description of an approach to compute mod 44 on Boolean shares, and detailing how this can be used to construct an improved decompose algorithm for masked Dilithium.

Masking Gadgets

[0081] Masking protects an intermediate variable x against side-channel attacks by splitting it into d shares and replacing all manipulations on x by manipulations on the d shares. This requires two types of masking, namely arithmetic masking and Boolean masking.

[0082] For arithmetic masking, a sensitive variable $x \in$ custom-character.sub.p is split into d shares x.sub.i.sup.A.sup.p$\in$custom-character.sub.p, $0 \le i < d$ with

[00005] $x = \sum_{i=0}^{d-1} x_i^{A_p} \bmod p$.

[0083] The ensemble of d shares of x is denoted as the arithmetic sharing x.sup.A.sup.p$\in$ custom-character.sub.p.sup.d.

[0084] Boolean masking enables protection of a k-bit variable x. The ensemble of the d shares of x is denoted as the Boolean sharing x.sup.B,k and the i-th share is denoted as x.sub.i.sup.B,k. The sharing of bits j.sub.1 to j.sub.2 of x is denoted as x.sup.B,k[j.sub.2:j.sub.1]. The relation between x and its shares is given as

[00006] $x = \bigoplus_{i=0}^{d-1} x_i^{B,k}$,
where ⊕ denotes a bitwise exclusive or (XOR).
[0085] In the following, the basic gadgets used in the algorithms are briefly described. The method described herein is independent of how these gadgets are implemented, provided the required security properties are fulfilled.
[0086] (+,−): Arithmetic addition and subtraction modulo q. A public constant is added only to the first share of an arithmetic sharing, e.g., for w.sup.A.sup.q+γ.sub.2 mod q. For operations on two arithmetic sharings, the operation is applied sharewise, e.g., for w.sup.A.sup.p−α.Math.w.sub.1.sup.A.sup.q mod q. [0087] (·): Arithmetic multiplication modulo q. A public constant is multiplied to each share of an arithmetic sharing independently, e.g., for α.sup.−1.Math.b.sup.A.sup.q. [0088] x: Boolean shares of a bit are multiplied with a public constant. This can be achieved in multiple ways, for example the bit shares are duplicated and combined via a secure AND function with the public constant. [0089] SecA2BModp.sub.q.sup.d(x.sup.A.sup.q): Converts an arithmetic input sharing x.sup.A.sup.q to a Boolean output sharing x.sup.B,k with q<2.sup.k. There are multiple known approaches, for example disclosed by Bronchain et al. [2]. [0090] SecB2AModp.sub.q.sup.d(x.sup.B,k): Converts a Boolean input sharing x.sup.B,k to an arithmetic output sharing x.sup.A.sup.q. There are multiple known approaches, for example disclosed by Bronchain et al. [2]. [0091] SecCompress.sub.q,−α.sub.−1.sup.d(x.sup.A.sup.q): Produces a compressed Boolean sharing y.sup.B,k′ of the arithmetic input sharing x.sup.A.sup.q. Azouaoui et al. [1] propose to use this operation to extract, w.sub.1.sup.B,k′ from w.sup.A.sup.q, in particular to securely divide a given shared input by −α. [0092] SecUnmask.sub.k′.sup.d(x.sup.B,k): Securely unmasks the Boolean input sharing x.sup.B,k. There are multiple known approaches, for example from Azouaoui et al. in [1]. [0093] SecAdd.sup.d(x.sup.B,k, y.sup.B,k), SecSub.sup.d(x.sup.B,k, y.sup.B,k): Computes arithmetic addition or subtraction of two Boolean input sharing x.sup.B,k and y.sup.B,k. [0094] SecAddModp.sub.q.sup.d(x.sup.B,k, y.sup.B,k): Computes arithmetic addition mod q of two Boolean input sharing x.sup.B,k and y.sup.B,k.
[0095] The basic building g blocks for the SecAdd.sup.d(x.sup.B,k, y.sup.B,k) and SecSub.sup.d(x.sup.B,k, y.sup.B,k) operations are masked full adders denoted as SecFullAdder.sup.d, which is described in Algorithm 1 below. Given two masked input bits (x.sup.B,1, y.sup.B,1) and a masked carry-in z.sup.B,1, the algorithm securely computes the sum w=x+y+z in a masked fashion, outputting the masked sum bit w.sup.B,2[0] and the masked carry-out w.sup.B,2[1]. The implementations described herein may be independent of the actual instantiation of this gadget. The gadget is required to fulfil the PINI security property and to produce the masked sum bit with zero latency. One such instantiation is given in Algorithm 2 based on the software-oriented implementation, where SecAnd.sub.1.sup.d denotes the PINI (Probe Isolating Non-Interference)-secure bitwise AND of two Boolean masked variables x.sup.B,k and y.sup.B,k.
[0096] These 1-bit full adders can then be chained to perform an addition of k-bit variables, commonly denoted as a ripple-carry adder. Algorithm 2 represents an exemplary instantiation of such a masked ripple-carry adder SecAdd.sub.k.sup.d, which is in line with prior descriptions of such a primitive, e.g., for low security orders. Algorithm 3 represents an exemplary instantiation of a corresponding subtraction operation SecSub.sub.k.sup.d. These secure addition and subtraction operations are used in the further algorithms described below.
TABLE-US-00001 Algorithm 1 - SecFullAdder.sup.d Input: Boolean sharing x.sup.B,1, y.sup.B,1, and z.sup.B,1. Output: Boolean sharing w.sup.B,2 such that w = x + y + z. 1: a.sup.B,1 ← x.sup.B,1 ⊕.sup.B y.sup.B,1 2: w.sup.B,2[0] ← z.sup.B,1 ⊕.sup.B a.sup.B,1 3: w.sup.B,2[1] ← x.sup.B,1 ⊕.sup.B SecAnd.sub.1.sup.d(a.sup.B,1, x.sup.B,1 ⊕.sup.B z.sup.B,1)

custom-character PINI SecAnd

TABLE-US-00002 Algorithm 2 - SecAdd.sub.k.sup.d Input: Boolean sharing x.sup.B,k and y.sup.B,k, such that x, y ∈ custom-character 0, 2.sup.k custom-character . Output: Boolean sharing z.sup.B,k such that z = x + y mod 2.sup.k. 1: c.sup.B,1 ← (0,0, ... ,0) 2: for i = 0 to k − 2 do 3: t.sup.B,2 ← SecFullAdder.sup.d (x.sup.B,k [i], y.sup.B,k [i], c.sup.B,1) custom-character Algorithm 1 4: (z.sup.B,k [i], c.sup.B,1) ← (t.sup.B,2 [0], t.sup.B,2 [1]) 5: z.sup.B,k [k − 1] ← x.sup.B,k [k − 1] ⊕.sup.B y.sup.B,k [k − 1] ⊕.sup.B c.sup.B,1

TABLE-US-00003 Algorithm 3 - SecSub.sub.k.sup.d Input: Boolean sharing x.sup.B,k and y.sup.B,k, such that x, y ∈ custom-character 0, 2.sup.k custom-character . Output: Boolean sharing z.sup.B,k such that z = x − y mod 2.sup.k. 1: c.sup.B,1 ← (1,0, ... ,0) 2: for i = 0 to k − 2 3: t.sup.B,2 ← SecFullAdder.sup.d (x.sup.B,k [i], y.sup.B,k [i], c.sup.B,1) custom-character Algorithm 1 4: (z.sup.B,k [i], c.sup.B,1) ← (t.sup.B,2 [0], t.sup.B,2 [1]) 5: z.sup.B,k [k − 1] ← x.sup.B,k [k − 1] ⊕.sup.B y.sup.B,k [k − 1] ⊕.sup.B c.sup.B,1

[0097] Algorithms 1, 2 and 3 above are described in co-pending U.S. application Ser. No. 18/298,100, filed Apr. 10, 2023, the content of which is incorporated herein by reference.

Decompose Operation

[0098] Dilithium requires the computation of a decompose operation that produces a tuple (w.sub.0, w.sub.1) given input w such that w=αw.sub.1+w.sub.0 mod q. One of the recent proposals from [1] to mask this operation is given in Algorithm 4 below. This takes as input an arithmetic sharing w.sup.A.sup.q and returns an arithmetic sharing w.sub.0.sup.A.sup.q and an unmasked integer w.sub.1. The main drawback of this approach is that it requires significantly different implementations for the parameter levels. In particular, NIST Level 2 is built on the complex SecCompress function. This results in increased implementation and code size overhead.

TABLE-US-00004 Algorithm 4: SecDecompose.sub.q,α.sup.d (w.sup.Aq) (from reference [1]) Input: Arithmetic sharing w.sup.A.sup.q, prime integer q with q < 2.sup.k integer α with 0 < α < q and α = 2γ.sub.2. Output: Arithmetic sharing w.sub.0.sup.A.sup.q and integer w.sub.1 such that w = αw.sub.1 + w.sub.0 mod q. 1: if NIST Level 3 or Level 5 then 2: b.sup.Aq ← w.sup.Aq + γ.sub.2 mod q 3: b′.sup.A.sup.q ← α.sup.−1 .Math. b.sup.A.sup.q − 1 mod q 4: b′.sup.B,k ← SecA2BModp.sub.q.sup.d(b′.sup.Aq) 5: w.sub.1.sup.B,k′ ← b′.sup.B,k [0 : k ′ − 1] 6: else (NIST Level 2) 7: w.sub.1.sup.B,k′ ← SecCompress.sub.q,−α.sub.−1.sup.d (w.sup.Aq) 8: w.sub.1 ← SecUnMask.sub.k′.sup.d (w.sub.1.sup.B,k′) 9: w.sub.0.sup.Aq ← w.sup.Ap − α .Math. w.sub.1 mod q

New Gadget: SecMod44

[0099] The aforementioned issue can be overcome by proposing an efficient and secure approach to compute mod 44 on Boolean shares with standard masking gadgets. An example of this method is described in Algorithm 5 below. The method, termed SecMod44, only requires bit shifts and the standard masking gadgets SecAdd and SecSub, described above. The approach is designed to be implemented in bit-sliced fashion, which allows for efficient operations on bit ranges as necessary. In each range, the effective bit size of the variables decreases, which in effect reduces the complexity of the masked addition and subtraction gadgets. The algorithm works as a sequence of congruences modulo 44, followed by two conditional additions depending on the sign bit of the result. The conditional additions are implemented using the X operations that masks the value by the conditional bit.

[0100] In general terms, in each step of the algorithm a relationship is used of the type a) 2.sup.i mod 44=2.sup.j or type b) 2.sup.i mod 44=44−2.sup.j. For mod 44, these relationships include for example 2.sup.22 mod 44=2.sup.2, 2.sup.12 mod 44=2.sup.2, 2.sup.17 mod 44=44−2.sup.2, and 2.sup.7 mod 44=44−2.sup.2. From this it follows that t[n:0]=t[i:0]+t[n:(i+1)]<<j (mod 44) for type a) and t[n:0]=t[i:0]−t[n:(i+1)]<<j (mod 44) for type b). These relationships can therefore be used to more efficiently decompose a Boolean share input mod 44.

[0101] A graphical representation of the algorithm, using an example of a 32 bit input Boolean share b′.sup.B,k, is illustrated in FIGS. **1** and **2**.

[0102] In Lines 1 and 2, illustrated in FIG. **1**, the algorithm uses the fact that 2.sup.12≡4 mod 44. As a result, the top 20 bits of the 32 bit Boolean share input, b′.sup.B,k [**31**:**12**], can be taken and left shifted by two bits, i.e. multiplied by 4, and added to the bottom 12 bits, i.e. b′.sup.B,k [**11**:**0**] using the SecAdd.sup.d addition operation. After Line 1, only 22 of the remaining bits are non-zero, indicated as t.sub.0.sup.B,22.

[0103] In line 2, bits **21**:**12** of the remaining 22 bits, t.sub.0.sup.B,22 [**21**:**12**], are left shifted by two bits, i.e. multiplied by 4, and added to the lower 12 bits, t.sub.0.sup.B,22 [**11**:**0**], again with SecAdd.sup.d, which results in a further reduction to t.sub.1.sup.B,12. After Line 2, only 12 bits remain.

[0104] In Lines 3 and 4, illustrated in FIG. **2**, the congruence 2.sup.7≡−4 mod 44 is used. In Line 3 the top 5 bits of t.sub.1.sup.B,12, i.e. t.sub.1.sup.B,12 [**11**:**7**], are left shifted by 2 bits and subtracted from the bottom 7 bits t.sub.1.sup.B,12 [**6**:**0**] using SecSub.sub.d. To avoid any borrows, this is done by first subtracting t.sub.1.sup.B,12 [**12**:**7**] from 264 (100001000) and adding the (positive) result to the bottom bits.

[0105] After Line 3, the result t.sub.2.sup.B,9 is already much reduced to the 9-bit interval [0, 391]. The same congruence 2.sup.7≡−4 mod 44 is then used in Line 4, which now allows borrows and adds 44 so that the result t.sub.3.sup.B,8 is within the interval [−56, 83]. Borrows are not an issue here through using two's complement representation for signed values.

[0106] In Line 5 we subtract 44 if the result is positive and add 44 if the result is negative. Afterwards, the result is in the interval [−12, 39]. Finally in line 6, 44 is added if t.sub.4.sup.B,7 is negative to put the result w.sub.1.sup.B,k′ in the interval [0, 43].

TABLE-US-00005 Algorithm 5: SecMod44.sub.k′.sup.d (b′.sup.B,k) Input: Boolean sharing b′.sup.B,k, prime integer q = 2.sup.23 − 2.sup.13 + 1 with q < 2.sup.k. Output: Boolean sharing w.sub.1.sup.B,k′ such that w.sub.1 = b′ mod 44. 1: t.sub.0.sup.B,22 = SecAdd.sup.d (b′.sup.B,k [11: 0], b′.sup.B,k [31: 12] << 2) 2. t.sub.1.sup.B,12 = SecAdd.sup.d (t.sub.0.sup.B,22 [11: 0], t.sub.0.sup.B,22 [21: 12] << 2) 3. t.sub.2.sup.B,9 = SecAdd.sup.d (t.sub.1.sup.B,12 [6: 0], SecSub.sup.d (264, t.sub.1.sup.B,12 [11: 7] << 2)) 4. t.sub.3.sup.B,8 = SecSub.sup.d (t.sub.2.sup.B,9 [6: 0], SecAdd.sup.d (44, t.sub.2.sup.B,9 [8: 7] << 2)) 5. t.sub.4.sup.B,7 = SecSub.sup.d (t.sub.3.sup.B,8 [7: 0], SecSub.sup.d (44, t.sub.3.sup.B,8 [7] × 88)) 6. w.sub.1.sup.B,k′ = SecAdd.sup.d (t.sub.4.sup.B,7 [6: 0], t.sub.4.sup.B,7 [6] × 44)

[0107] The flow diagram in FIG. **3** illustrates the above description of the algorithm. The input b′.sup.B,k is provided at step **301**, which is subjected to the SecAdd.sup.d operation at step **302**, corresponding to Line 1, resulting in the 22-bit output t.sub.0.sup.B,24 at step **303**. This is subjected to the further SecAdd.sub.d operation at step **304**, corresponding to Line 2, resulting in the 13-bit output t.sub.1.sup.B,13 at step **305**. This result is then subjected to the further SecAdd.sup.d operation at step **306**, corresponding to Line 3, resulting in the 9-bit output t.sub.2.sup.B,9 at step **307**. Step **308** then applies the SecSub.sup.d of Line 4 to further reduce to the 8-bit output t.sub.3.sup.B,8 at step **309**. At step **310**, the output t.sub.3.sup.B,8 is determined to be either positive (+ve) or negative (−ve). If positive, 44 is subtracted at step **312** and if negative 44 is added at step **311**, resulting in the 7-bit output t.sub.4.sup.B,7 at step **313**. At step **314**, the output t.sub.4.sup.B,7 is determined to be either positive or negative. If negative, 44 is added at step **315**. The output is then the decomposed output w.sub.1.sup.B,k′ at step **316**, which is a mod 44 representation of the input b′.sup.B,k.

[0108] Using the gadget SecMod44, the existing decompose given in Algorithm 4 above can be adapted, resulting in the proposed masked decompose given in Algorithm 6 below. The two possible cases (i.e., levels 3 and 5 and level 2) share most of the operations and only differ in the masked modulo computation.

[0109] Although a 32-bit input value is indicated in the example described herein, it should be understood that the method may be applied to inputs having a different number of bits, provided the number of bits is greater than 12. Steps **302** and **304** in effect repeat the same type of process to

reduce an initial 32-bit input to a 12-bit intermediate output t.sub.1.sup.B,12. In alternative implementations having different numbers of bits in the input value, these steps may be implemented in a single process or more than two steps in order to reduce the initial number of bits to an output having 12 bits.

[0110] For levels 3 and 5, an existing approach can be reused using the efficient mod 16 computation on Boolean shares. For level 2, the SecMod44 gadget can be used. Since this can be easily implemented with existing standard gadgets, the improved decompose has a lower implementation and code size complexity than the previous solutions. Azouaoui et al. [1] also propose a slightly modified decompose that does not unmask w.sub.1. The proposed algorithm is also compatible with this approach. Line 8 in Algorithm 6 would need to be replaced by a SecB2AModp.sub.q.sup.d(w.sub.1.sup.B,k′) conversion, and Line 9 would need to be adapted accordingly to process an arithmetically shared w.sub.1.

TABLE-US-00006 Algorithm 6: SecDecompose − New.sub.q,α.sup.d (w.sup.A.sup.q) Input: Arithmetic sharing w.sup.A.sup.q, prime integer q with q < 2.sup.k, integer α with 0 < α < q and α = 2γ.sub.2. Output: Arithmetic sharing w.sub.0.sup.A.sup.q , Boolean sharing w.sub.1.sup.B,k′ such that w = αw.sub.1 + w.sub.0 mod q. 1: b.sup.A.sup.q ← w.sup.A.sup.q + γ.sub.2 mod q 2. b′.sup.A.sup.q ← a.sup.−1 .Math. b.sup.A.sup.q − 1 mod q 3. b′.sup.B,k ← SecA2BMod.sub.q.sup.d (b′.sup.A.sup.q) 4. if NIST Level 3 of Level 5 then 5. w.sub.1.sup.B,k′ ← b′.sup.B,k [0: k′ − 1] 6. else (NIST Level 2) 7. w.sub.1.sup.B,k′ ← SecMod44.sub.k.sup.d (b′.sup.B,k) 8. w.sub.1 ← SecUnMask.sub.k′.sup.d (w.sub.1.sup.B,k′) 9. w.sub.0.sup.A.sup.q ← w.sup.A.sup.q − α .Math. w.sub.1 mod q

[0111] FIG. **4** is a schematic diagram illustrating a method of performing a signing operation in a Dilithium digital signature scheme. An input **401** to be digitally signed is received by a hardware digital signing system **402**, which may for example be incorporated into an integrated circuit in an article such as a smart card. The digital signing system **402** performs a digital signing operation on the received input **401** using a secret key stored within the system **402**. A signature output **403** is provided from the digital signing operation performed by the system **402**. An attacker **404** attempting to perform a side-channel attack on the system **402** is unable to extract the secret key from the system **402** due to the use of the digital signature scheme performed, which involves a masking and decomposition process as described herein.

[0112] FIG. **5** is a schematic diagram illustrating an example hardware converter **500** configured to decompose mod 44 an N bit Boolean share input b′.sup.B,k to provide an output w.sub.1.sup.B,k′. The hardware converter **500** comprises a first bit reduction module **501**, a second bit reduction module **502** and an adjustment module **503**. The first bit reduction module **501** is configured to reduce a number of bits in the Boolean share input b′.sup.B,k by adding a lower 11:0 bits of the input to an upper portion of the input left shifted by 2 bits to provide a first intermediate result having M bits. The second bit reduction module **502** is configured to reduce a number of bits of the first intermediate result by adding a lower 6:0 portion of the intermediate result to an upper portion left shifted by 2 bits and subtracted from a multiple of 44 to provide a second intermediate result. The adjustment module **503** is configured to adjust the second intermediate result by adding and/or subtracting 44 to provide an output w.sub.1.sup.B,k′ having a value within an interval of 0:43, the output being a mod 44 representation of the input b′.sup.B,k. Further optional features of the hardware converter **500** may be described with reference to the corresponding features of the example method as described above.

[0113] From reading the present disclosure, other variations and modifications will be apparent to the skilled person. Such variations and modifications may involve equivalent and other features which are already known in the art of cryptographic methods in general and demasking operations in Dilithium in particular, and which may be used instead of, or in addition to, features already described herein.

[0114] Although the appended claims are directed to particular combinations of features, it should

be understood that the scope of the disclosure of the present invention also includes any novel feature or any novel combination of features disclosed herein either explicitly or implicitly or any generalisation thereof, whether or not it relates to the same invention as presently claimed in any claim and whether or not it mitigates any or all of the same technical problems as does the present invention.

[0115] Features which are described in the context of separate embodiments may also be provided in combination in a single embodiment. Conversely, various features which are, for brevity, described in the context of a single embodiment, may also be provided separately or in any suitable sub-combination. The applicant hereby gives notice that new claims may be formulated to such features and/or combinations of such features during the prosecution of the present application or of any further application derived therefrom.

[0116] For the sake of completeness it is also stated that the term “comprising” does not exclude other elements or steps, the term “a” or “an” does not exclude a plurality, a single processor or other unit may fulfil the functions of several means recited in the claims and reference signs in the claims shall not be construed as limiting the scope of the claims.

REFERENCES

[0117] [**1**] Melissa Azouaoui et al, “Protecting dilithium against leakage: Revisited sensitivity analysis and improved implementations”, IACR Cryptol. ePrint Arch. (2022), 1406. [0118] [**2**] Olivier Bronchain and Gaëtan Cassiers, “Bitslicing arithmetic/boolean masking conversions for fun and profit with application to lattice-based kems”, IACR Trans. Cryptogr. Hardw. Embed. Syst. 2022 (2022), no. 4, 553-588. [0119] [**3**] Jean-Sébastien Coron et al, “High-order masking of ntru”, IACR Transactions on Cryptographic Hardware and Embedded Systems 2023 (2023), no. 2, 180-211. [0120] [**4**] Jean-Sébastien Coron et al, “Improved gadgets for the high-order masking of dilithium”, Cryptology ePrint Archive, Paper 2023/896, 2023, https://eprint.iacr.org/2023/896. [0121] [**5**] L. Ducas et al, Crystals-dilithium algorithm specifications and supporting documentation (version 3.1), 2021. [0122] [**6**] Vincent Migliore et al, “Masking dilithium-efficient implementation and side-channel evaluation”, Applied Cryptography and Network Security-17th International Conference, ACNS 2019, Bogota, Colombia, Jun. 5-7, 2019, Proceedings (Robert H. Deng, Valérie Gauthier-Umaña, Martín Ochoa, and Moti Yung, eds.), Lecture Notes in Computer Science, vol. 11464, Springer, 2019, pp. 344-362. [0123] [**7**] National Institute of Standards and Technology, Post-quantum cryptography standardization. https://csrc.nist.gov/Projects/Post-Quantum-Cryptography/Post-Quantum-Cryptography-Standardization. [0124] [**8**] Matthieu Rivain and Emmanuel Prouff, “Provably secure higher-order masking of AES, Cryptographic Hardware and Embedded Systems”, CHES 2010, 12th International Workshop, Santa Barbara, CA, USA, Aug. 17-20, 2010. Proceedings (Stefan Mangard and François-Xavier Standaert, eds.), Lecture Notes in Computer Science, vol. 6225, Springer, 2010, pp. 413-427.

Claims

1-**14**. (canceled)

15. A hardware converter configured to decompose, using a mod 44 operation, an N bit Boolean share input, where N>12, the hardware converter comprising: a first bit reduction module configured to reduce a number of bits in the N-bit Boolean share input by adding a lower 11:0 bits of the N-bit Boolean share input to an upper portion of the N-bit Boolean input, which is left-shifted by 2 bits, to produce a first intermediate result having M bits; a second bit reduction module configured to reduce a number of bits of the first intermediate result by adding a lower 6:0 portion of the intermediate result to an upper portion left shifted by 2 bits and subtracted from a multiple of 44 to provide a second intermediate result; and an adjustment module configured to adjust the second intermediate result by adding or subtracting 44 to provide an output having a value within an interval of 0:43, the output being a mod 44 representation of the N bit Boolean share input.

16. The hardware converter of claim 15, wherein the first bit reduction module is configured to: reduce the number of bits in the Boolean share input by adding the lower 11:0 bits of the input to an upper N−1:12 portion of the input left shifted by 2 bits to provide a third intermediate result having P bits, where P<N; and reduce a number of bits in the third intermediate result by adding a lower 11:0 bits of the third intermediate result to an upper P−1:12 portion of the third intermediate result left shifted by 2 bits to provide the first intermediate result.
17. The hardware converter of claim 15, wherein the second bit reduction module is configured to: subtract the first intermediate result from an integer multiple of 44 to provide a fourth intermediate result; left shift an upper 6 bits of the fourth intermediate result by two bits and subtract this from a lower 7 bits of the fourth intermediate result to provide a fifth intermediate result; add 44 to the fifth intermediate result from 44 to provide a sixth intermediate result; and subtract a lower 6 bits of the sixth intermediate result to provide the second intermediate result.
18. The hardware converter of claim 17, wherein the integer multiple is 6.
19. The hardware converter of claim 15, wherein N=32.
20. The hardware converter of claim 15, wherein M=12.
21. A method of decomposing, using a mod 44 operation, an N bit Boolean share input, where N>12, the method comprising: reducing a number of bits in the N bit Boolean share input, using a first bit reduction module, by adding a lower 11:0 bits of the input to an upper portion of the input left shifted by 2 bits to provide a first intermediate result having M bits; reducing a number of bits of the first intermediate result, using a second bit reduction module, by adding a lower 6:0 portion of the intermediate result to an upper portion left shifted by 2 bits and subtracted from a multiple of 44 to provide a second intermediate result; and adjusting, using an adjustment module, the second intermediate result by adding and/or subtracting 44 to provide an output having a value within an interval of 0:43, the output being a mod 44 representation of the N bit Boolean share input.
22. The method of claim 21, wherein reducing the number of bits in the N bit Boolean share input, using the first bit reduction module, comprises: reducing the number of bits in the N bit Boolean share input by adding the lower 11:0 bits of the input to an upper N−1:12 portion of the N bit Boolean share input left shifted by 2 bits to provide a third intermediate result having P bits, where P<N; and reducing a number of bits in the third intermediate result by adding a lower 11:0 bits of the third intermediate result to an upper P−1:12 portion of the third intermediate result left shifted by 2 bits to provide the first intermediate result.
23. The method of claim 21, wherein reducing the number of bits of the first intermediate result, using the second bit reduction module, comprises: subtracting the first intermediate result from an integer multiple of 44 to provide a fourth intermediate result; left shifting an upper 6 bits of the fourth intermediate result by two bits and subtracting this from a lower 7 bits of the fourth intermediate result to provide a fifth intermediate result; adding 44 to the fifth intermediate result from 44 to provide a sixth intermediate result; and subtracting a lower 6 bits of the sixth intermediate result to provide the second intermediate result.
24. The method of claim 23, wherein the integer multiple is 6.
25. The method of claim 21, wherein N=32.
26. The method of claim 21, wherein M=12.
27. The method of claim 21, further comprising generating a digital signature based on a digital signature operation using the output and a secret key.
28. The computer-implemented method of claim 27, wherein the digital signature operation is a Dilithium digital signature operation.
29. A non-transitory computer readable medium storing processor-readable instructions that, when executed, cause a processor to decompose, using mod 44 operation, an N bit Boolean share input, where N>12, the non-transitory computer readable medium storing comprising the processor-readable instructions that cause the processor to: reduce a number of bits in the N bit Boolean share input by adding a lower 11:0 bits of the input to an upper portion of the input left shifted by 2 bits

to provide a first intermediate result having M bits; reduce a number of bits of the first intermediate result by adding a lower 6:0 portion of the intermediate result to an upper portion left shifted by 2 bits and subtracted from a multiple of 44 to provide a second intermediate result; and adjust the second intermediate result by adding or subtracting 44 to provide an output having a value within an interval of 0:43, the output being a mod 44 representation of the N bit Boolean share input.

30. The non-transitory computer readable medium of claim 29, wherein the processor-readable instructions that cause the processor to reduce a number of bits comprises processor-readable instructions that cause the processor to: reduce the number of bits in the N bit Boolean share input by adding the lower 11:0 bits of the input to an upper N−1:12 portion of the N bit Boolean share input left shifted by 2 bits to provide a third intermediate result having P bits, where P<N; and reduce a number of bits in the third intermediate result by adding a lower 11:0 bits of the third intermediate result to an upper P−1:12 portion of the third intermediate result left shifted by 2 bits to provide the first intermediate result.

31. The non-transitory computer readable medium of claim 29, wherein the processor-readable instructions that cause the processor to reduce the number of bits of the first intermediate result comprises processor-readable instructions that cause the processor: subtract the first intermediate result from an integer multiple of 44 to provide a fourth intermediate result; left shift an upper 6 bits of the fourth intermediate result by two bits and subtracting this from a lower 7 bits of the fourth intermediate result to provide a fifth intermediate result; add 44 to the fifth intermediate result from 44 to provide a sixth intermediate result; and subtract a lower 6 bits of the sixth intermediate result to provide the second intermediate result.

32. The non-transitory computer readable medium of claim 31, wherein the integer multiple is 6.

33. The non-transitory computer readable medium of claim 29, wherein N=32.